Snapshot of October 29, 2026. This dossier describes the four repositories as they stood that day; the projects have moved on since, and the repositories are the authority. Underlined links lead to the sources, frozen at their state of that day. What came next lives on gnu6dotlive.github.io.

DOSSIER 2 · THE ENGINE

gnostral.rs

An open lab trying to run large artificial-intelligence models on a modest graphics card — and publishing its measurements, including the disappointing ones.

RESEARCH | V0.8.0 | MIT LICENCE

2.12×
faster than the previous version of the **same engine**

20.18
tokens per second, against 9.51 at the start

22 → 13
layers placed on the card depending on free memory

3/3
model families served by the same executable

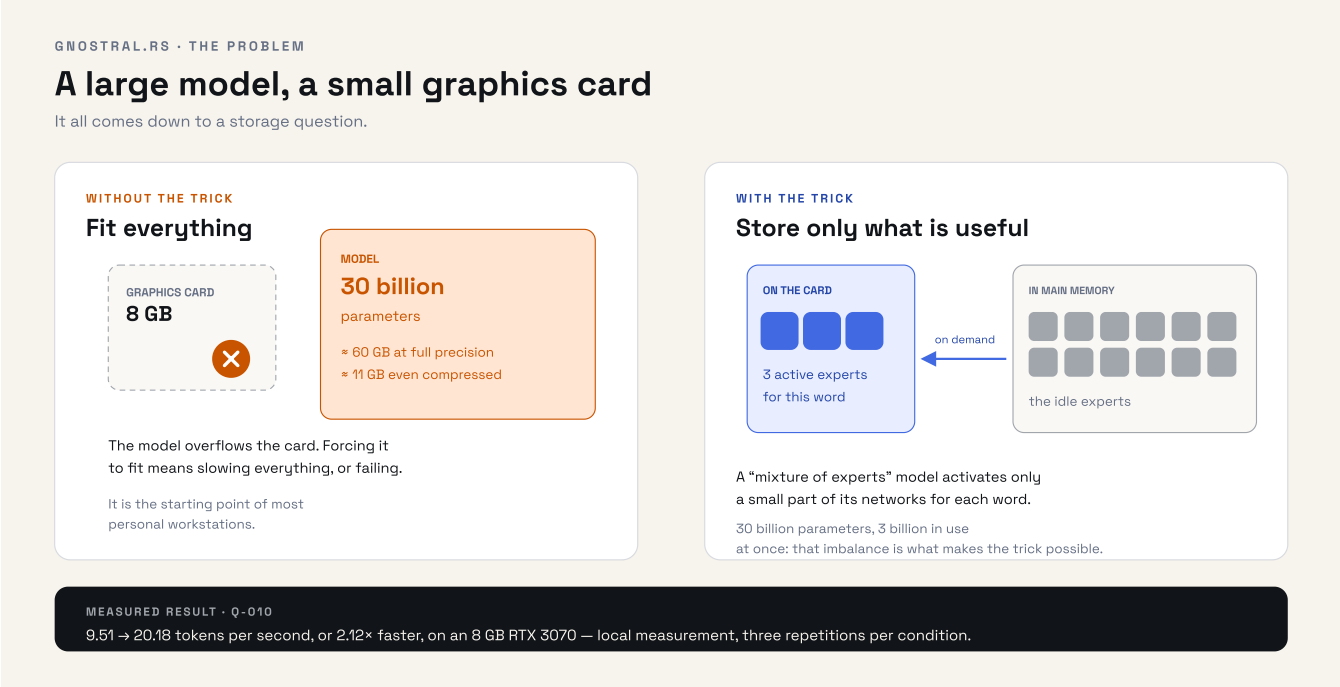
In one picture

THE ANALOGY

Picture a warehouse and a small counter. The **counter** is the graphics card: narrow, but very fast. The **warehouse** is main memory: huge, but slow to serve from.

A 30-billion-parameter model does not fit on the counter. The trick is to put only the three billion pieces needed right now on it, and fetch the others from the warehouse only when they are used. That is what gnostral.rs is about.

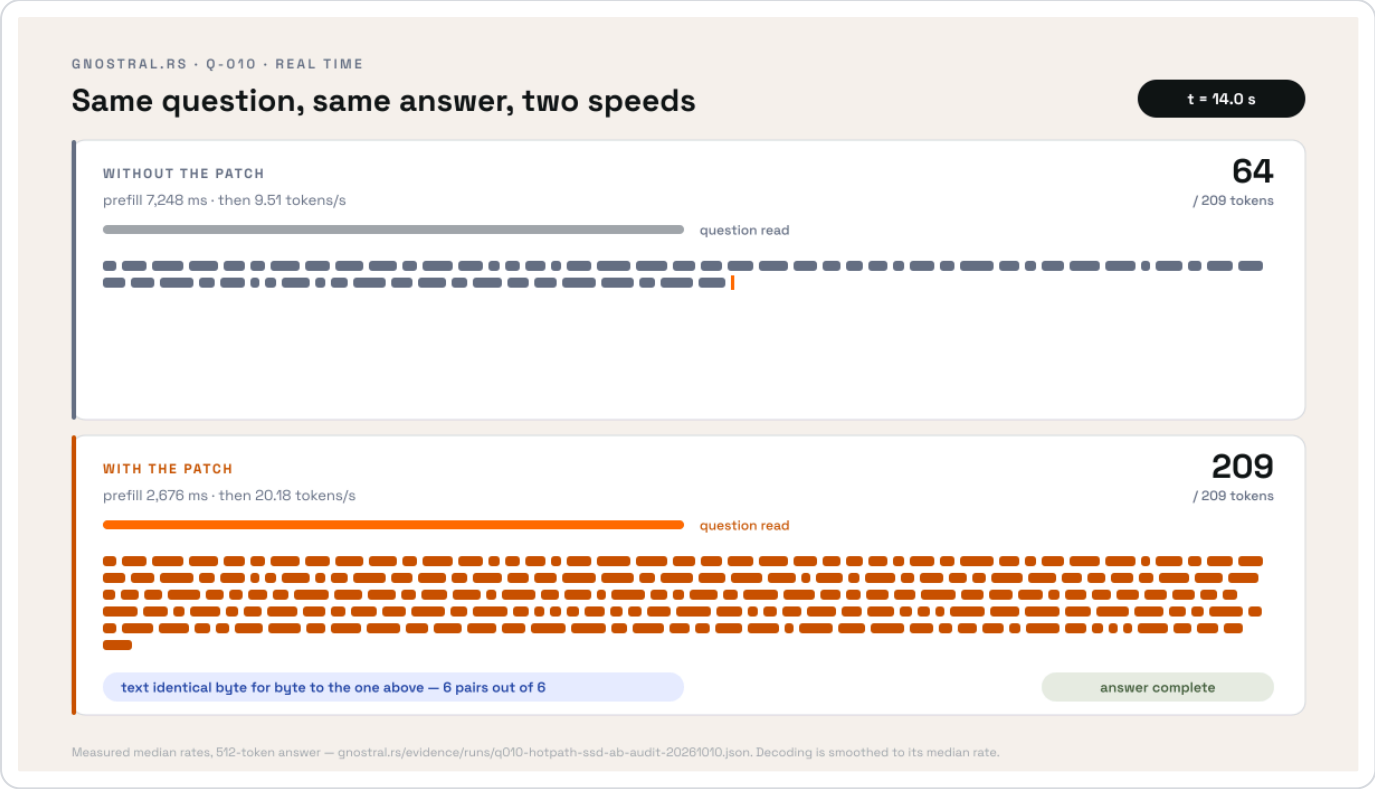
The problem, then the trick



A storage problem. A "mixture of experts" model activates only a small part of its networks for each word. That imbalance is what makes splitting it possible.

The reference machine is not a datacenter server: an RTX 3070 with 8 GB of graphics memory and 32 GB of main memory. It is a personal-workstation configuration — and that is exactly what makes the research interesting.

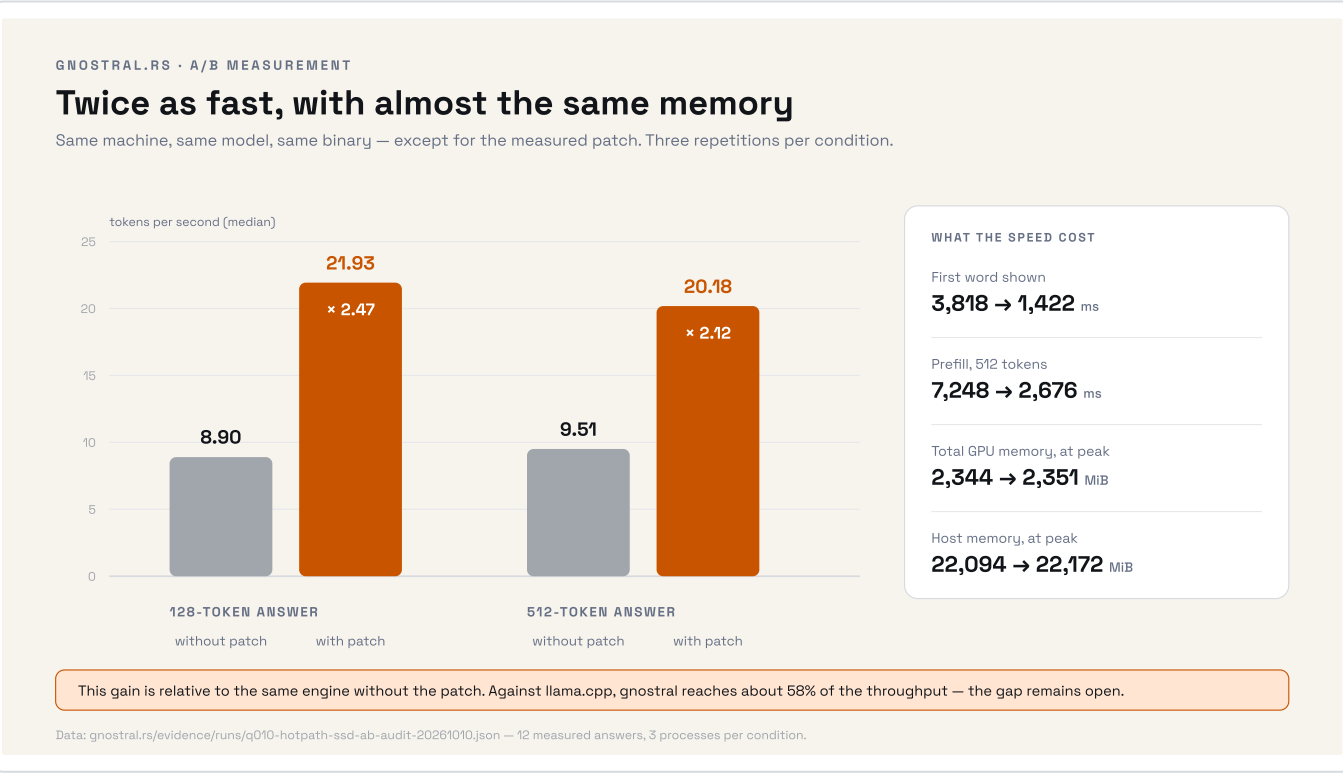
The headline result: by keeping the always-used parts on the card and borrowing the quantised experts from main memory, throughput goes from **9.51 to 20.18 tokens per second**, or 2.12 times faster. Measured locally, three repetitions per condition, responses identical byte for byte.



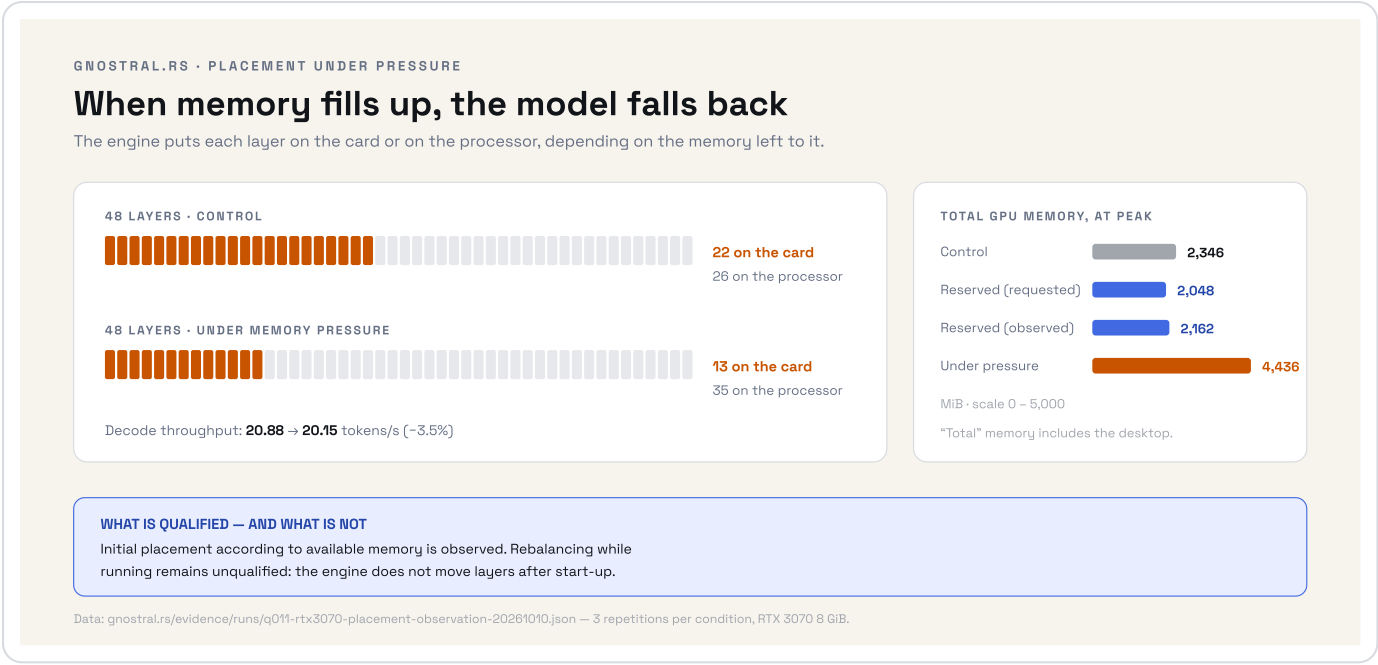
The same answer, at the measured pace. Both lanes write exactly the same text — verified byte for byte. Without the patch, reading the question takes 7.2 s, then 9.5 tokens per second; with it, 2.7 s, then 20.2 tokens per second. The time shown is real time.

What was measured — and within what frame

MILESTONE	RESULT	WHAT IT DOES NOT SAY
Borrowed experts Q-010	9.51 → 20.18 tokens/s on a 512-token answer, median of three repetitions, 2.12×	Not faster than llama.cpp (34.54 tokens/s on the same machine); not valid for every architecture.
Placement at load time Q-011	22/21/21 layers on the card → 13/12/13 under a memory reservation	Initial placement only; no migration while serving.
One engine, three families Q-012	3/3: dense model, embeddings, quantised MoE	Three successive servers; no co-residence, no production stability.
A real dense process Q-013-C	Exact semantic answer "GNOSTRAL"; +8 MiB of GPU memory after shutdown	A single model, a short-lived server, not a public API.



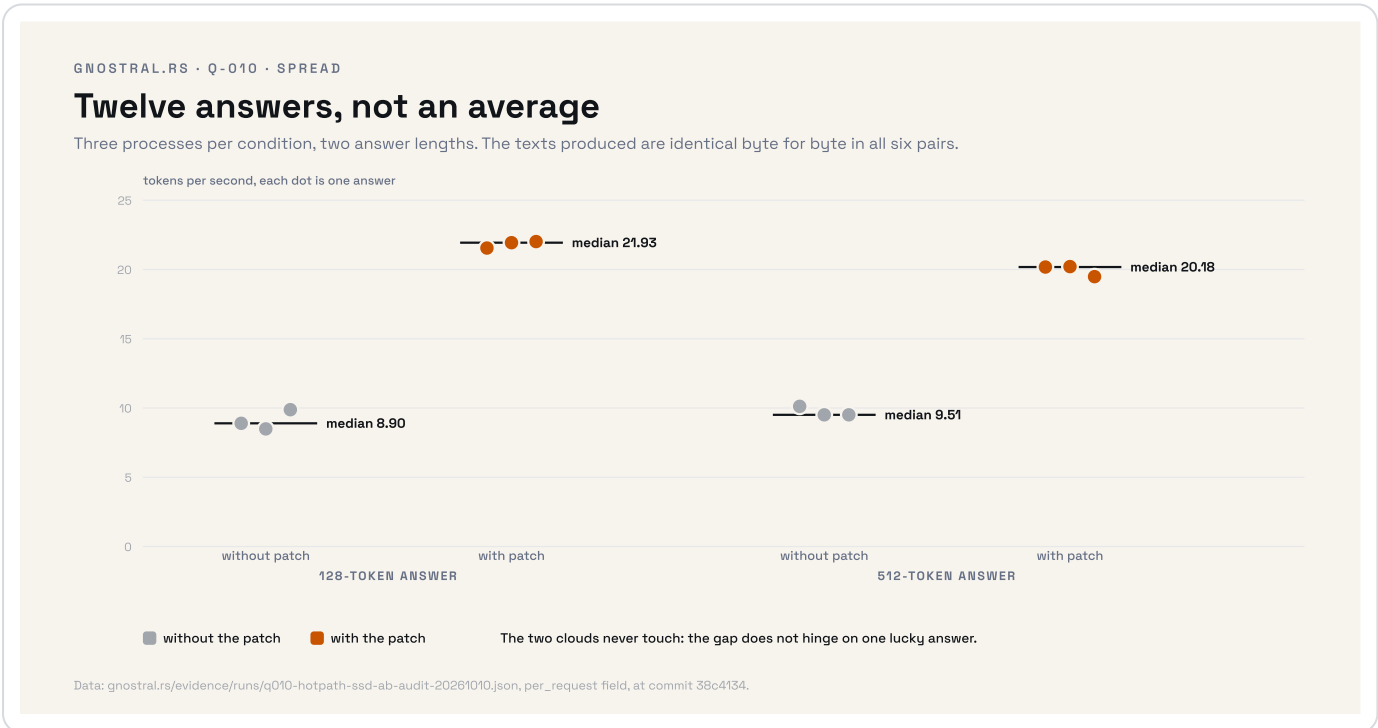
The gain, and its price. Same machine, same model, same binary except for the patch: throughput doubles, the first word arrives 2.7 times sooner, and graphics memory moves by only 7 MiB. The reference remains the same engine without the patch — not llama.cpp.



Placement, observed. When graphics memory is taken up before start-up, the engine puts fewer layers on the card (22 → 13 in these two runs) and throughput drops by only 3.5%. Moving layers while running, however, is not demonstrated.

Twelve answers, not an average

A "2.12×" is a median. Behind it are twelve measured answers: three processes without the patch and three with it, each asked for a 128-token then a 512-token answer. The two clouds never touch: the slowest patched answer (19.48 tokens/s) is still twice as fast as the fastest unpatched one (10.12).



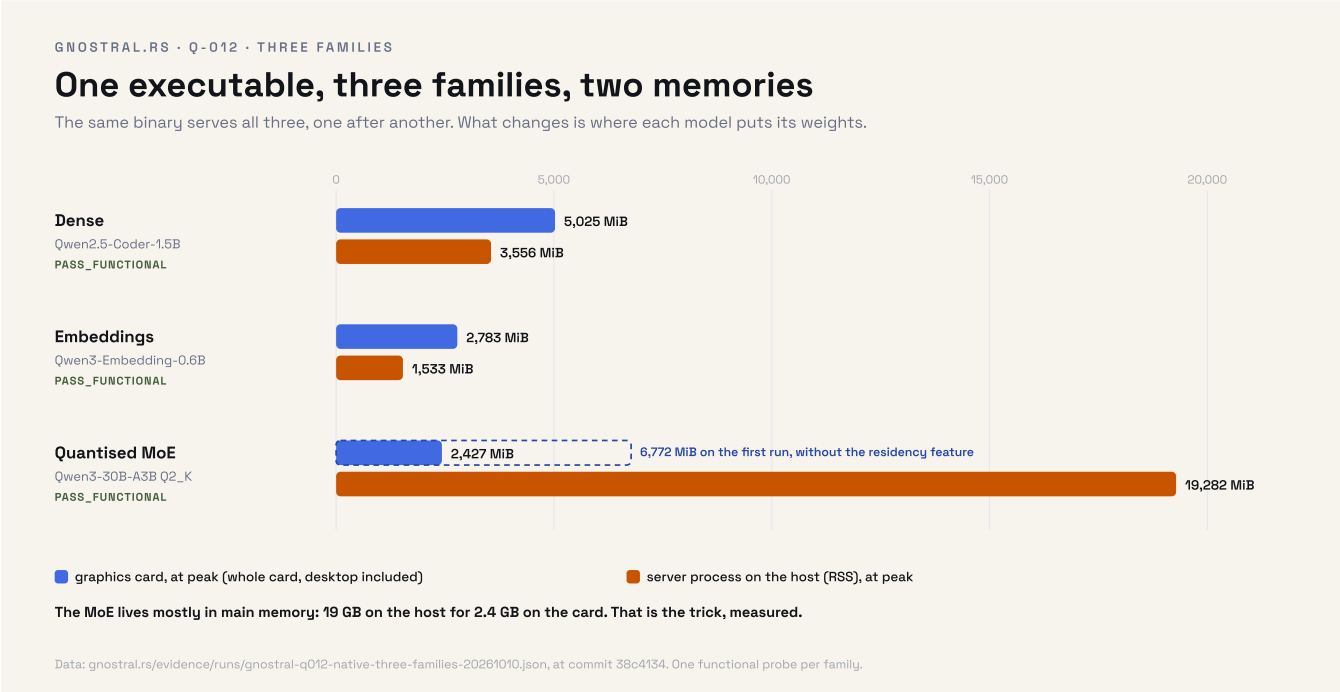
The gap does not hinge on one lucky answer. The texts produced are identical byte for byte in all six pairs: the patch speeds things up without changing anything that is written.

> THE DATA, ANSWER BY ANSWER

One executable, three model families

Milestone Q-012 asks another question: can the same executable serve models of different kinds? Three families, one after another: a dense model that writes text, an *embeddings* model that turns sentences into vectors, and the large quantised MoE. All three answer correctly — the embeddings model, for instance, rates two related sentences much closer (0.78) than two unrelated ones (0.17).

Above all, the chart shows **where each model puts its weights**. The 30-billion-parameter MoE takes 2.4 GB on the card and 19 GB in main memory: that is the dossier's trick, measured. The first run, built without the expert-residency feature, climbed to 6.8 GB on the card and timed out; it stays on record, and a guard now refuses a binary that forgets it.



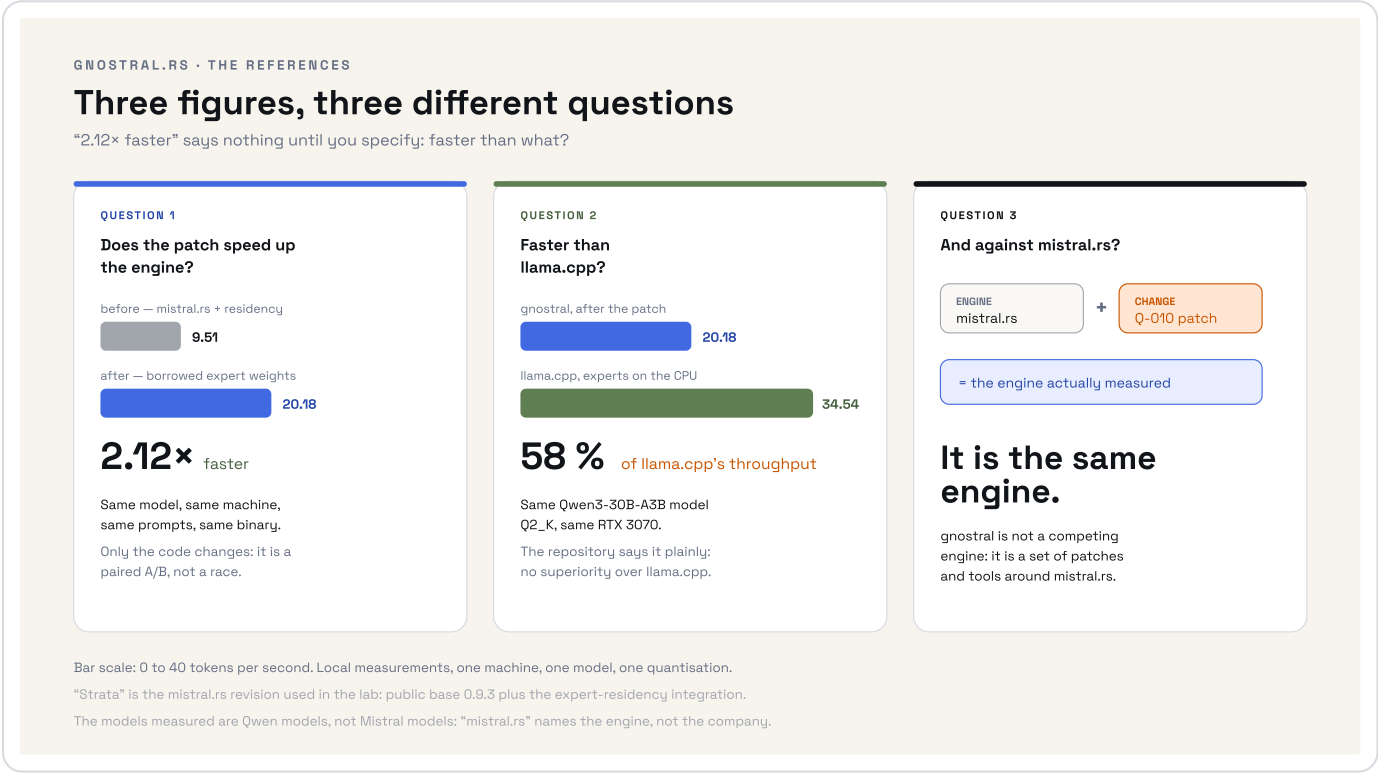
Two memories, two uses. Card memory is measured for the whole card, desktop included; host memory is that of the server process alone. One functional probe per family: this is not a quality benchmark.

REMEMBER

Every figure has its report, its protocol and its limits. The repository explicitly refuses to publish a figure without the evidence that goes with it — including when the evidence contradicts a more flattering claim.

Compared with what, exactly?

A "2.12× faster" means nothing without its point of comparison. Here, the reference is not another piece of software: it is **the same engine, one version earlier**.



Three references, three answers. The 2.12× gain is a paired A/B on the same engine. The comparison with llama.cpp is unfavourable. The comparison with mistral.rs makes no sense: it is the engine the patch modifies.

Concretely, "Strata" is the revision of mistral.rs used in the lab: public version 0.9.3, plus an expert-residency integration. The 9.51 tokens/s measure that engine *before* the patch; the 20.18 measure the same binary, rebuilt with a single function changed — the quantised expert weights are borrowed instead of being copied on every call.

Against llama.cpp, on the other hand, the comparison exists and it is unfavourable: **34.54 tokens/s for llama.cpp against 20.18 for gnostral**, on the same model and the same card. The repository's report says so explicitly: these gains do not establish superiority over llama.cpp's CPU-MoE options.

As for mistral.rs, it is not an opponent: it is the engine *in which* the patch is applied. gnostral is not a competing engine — it is a set of patches, typed contracts and measurement tools around mistral.rs. Comparing against it would be like comparing a car with its own engine.

MIND THE WORDS

mistral.rs is an inference engine written in Rust. **Mistral AI** is a company that makes models. The two are unrelated, and the models measured here are **Qwen** models, not Mistral models.

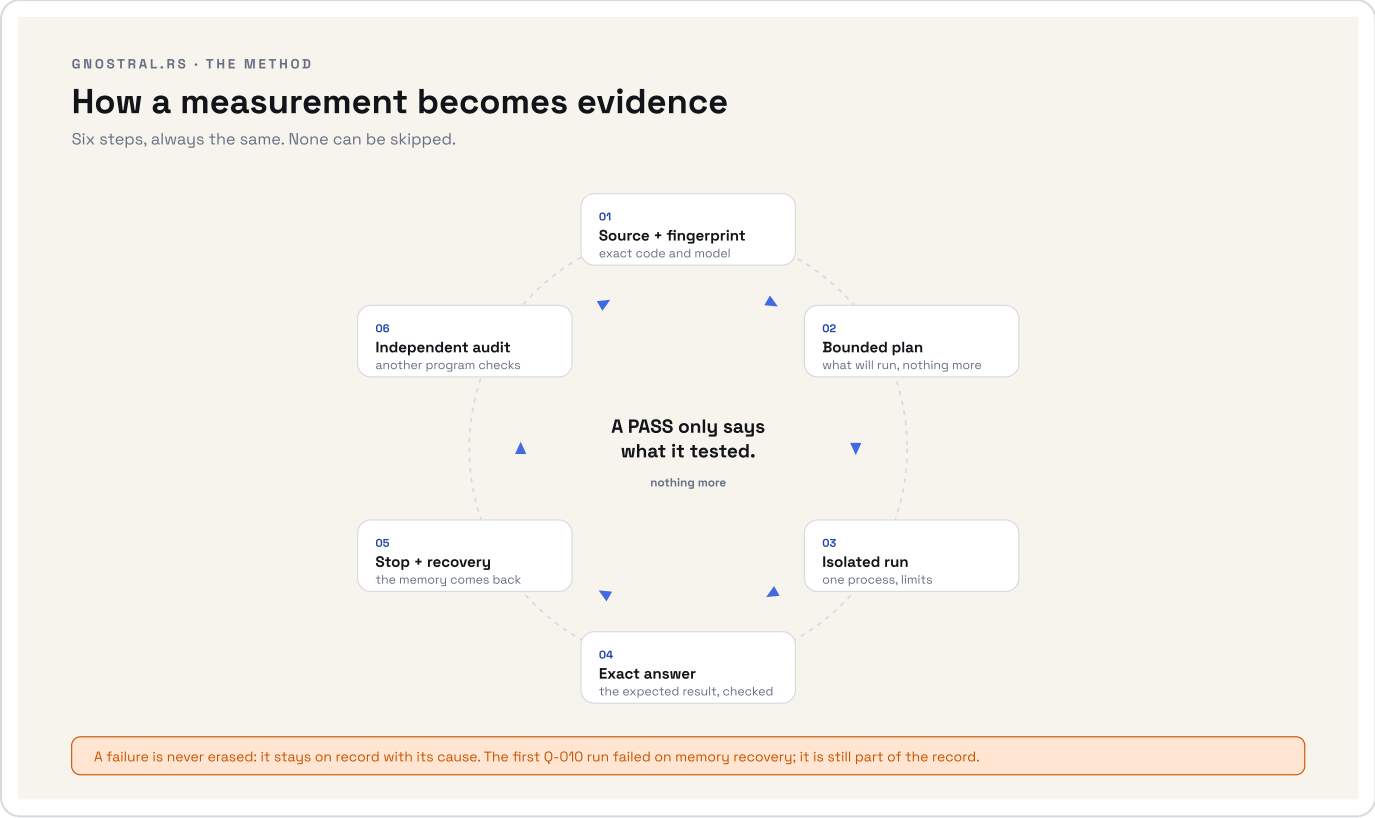
Saying "faster than Mistral" therefore makes no sense until you say which one you mean — and, if it is the engine, which version serves as the reference.

WHAT IT CHANGES

The exact wording is: "2.12× faster than the same mistral.rs revision without the patch, and still 58% of llama.cpp's throughput on this model". It is less spectacular, but it is true, dated and verifiable.

How a measurement becomes evidence

The project's most solid contribution may not be the speed gain, but the **method**. Six steps, always the same, and none can be skipped.



The qualification loop. A "PASS" only says what the test tested. It says nothing about stability, security or performance on another machine.

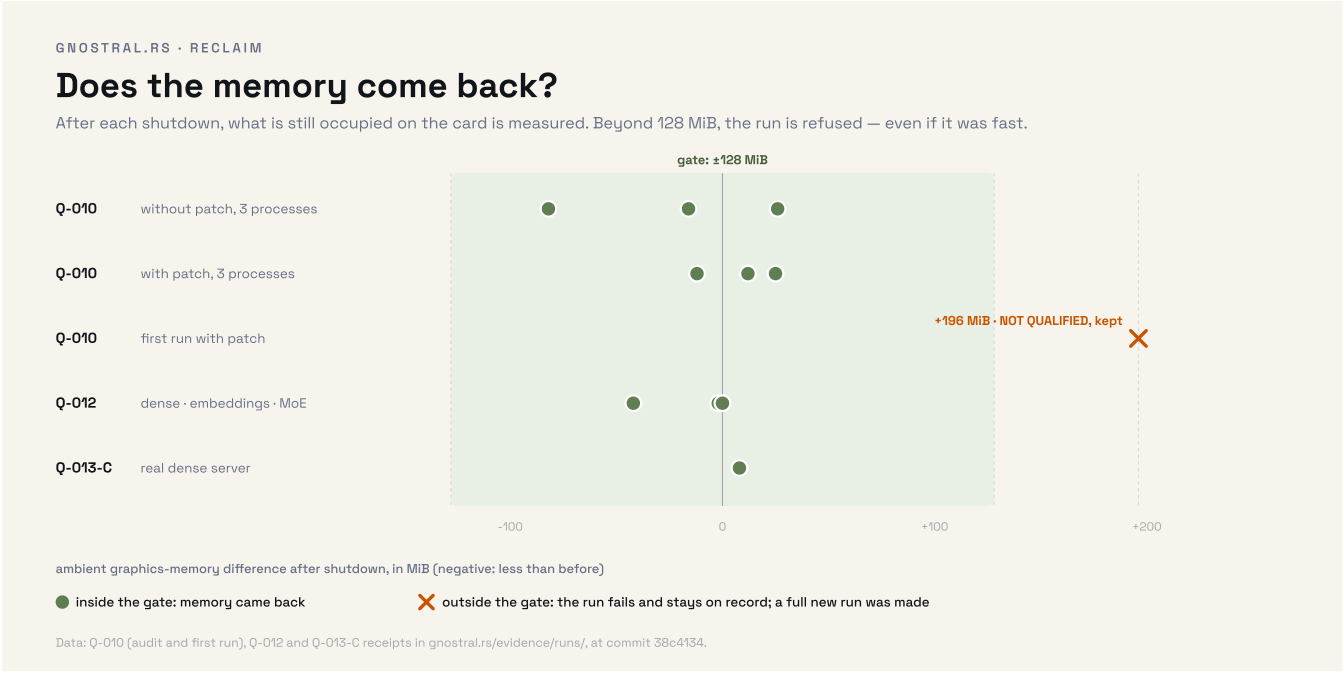
Concretely: the code and the model are identified by a fingerprint (SHA), a plan describes exactly what will run, a single process is started within limits, a precise answer is checked, the process is stopped, the memory is checked to have come back, and then an independent program redoes the checks.

Order matters. A server can answer a health check while its weights are not yet loaded. gnostral.rs therefore distinguishes eight states, from the mere fact of being "declared in the documentation" up to "qualified by an independent audit".

› THE EIGHT STATES OF TRUTH, PLAINLY

Does the memory come back?

The loop's sixth step — stopping and recovery — is the one that fails the most runs. After each shutdown, what is still occupied on the graphics card is measured. The gate is set *before* the run: **±128 MiB**. A fast run that leaves memory behind is refused.



A visible failure among the successes. The first patched Q-010 run left 196 MiB on the card — cause undetermined, possibly desktop GPU traffic. It is marked NOT QUALIFIED and kept; the published measurement comes from a new full session, with the same thresholds.

What it is not

THE LIMITS, PLAINLY

- **Not a server ready to deploy.** It is not a standalone replacement for mistral.rs, llama.cpp, Candle or vLLM.
- **No hot model switching.** The sequence A → B → A under a single supervisor has not been demonstrated.
- **No secured public API.** The driver talks to localhost on a fixed port, without authenticating a hostile peer.
- **No multi-user service.** No control plane or availability commitment has been ratified.
- **No guarantee on your hardware.** The figures come from one precise configuration and are explicitly not portable.

Why the failures are public

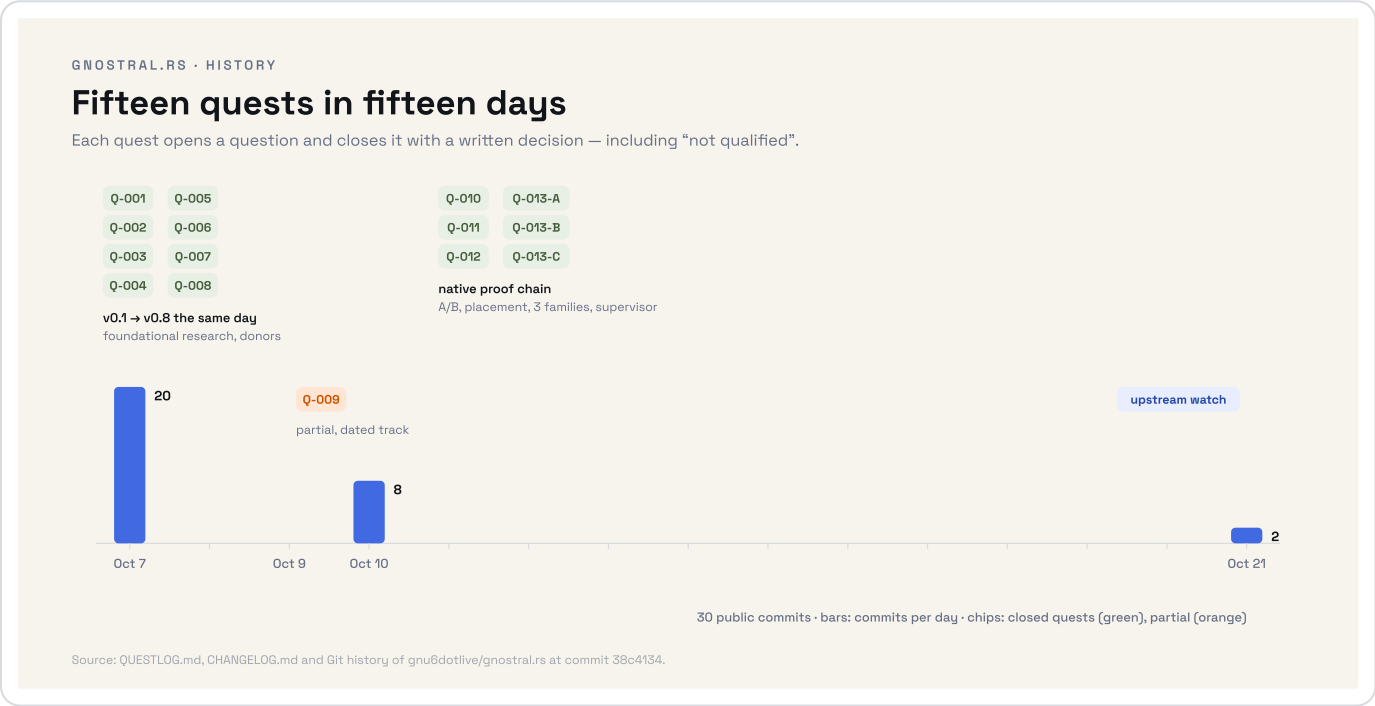
The first Q-010 run failed: graphics memory did not come back within the expected margin (+196 MiB against a ±128 MiB tolerance). The first Q-012 run failed too: a compile-time feature was missing, and the MoE model timed out.

Both failures are still on the record. It is a methodological choice, not an affectation: a result whose misses you do not know is not reusable. And a figure that cannot be contradicted is not a measurement; it is a claim.

The same principle applies to the Bonsai benchmark: the reference kept is 35.2 tokens per second, not the more flattering claim of 40 — for lack of sufficient evidence.

History: fifteen quests in fifteen days

gnostral.rs moves forward by **quests**: each opens a precise question and closes with a written decision, including when the decision is "not qualified". The public log counts fifteen between October 7 and 21, 2026.



Two days do most of the work. October 7 closes the foundational research and publishes versions 0.1 to 0.8; the 10th, the native proof chain, from the A/B to the real supervised server.

Projects studied, and turned down

Part of the work is examining other engines before touching them. The repository keeps its refusals as carefully as its adoptions:

xInfer — not qualified

A clean build failure against CUDA 13.4, model loads running out of memory. Adoption refused: the lab does not turn into a repair shop for an upstream project.

zLLM — blocked

A Gemma model served correctly, but the adaptive-placement promise could not be tested: the 17 to 19 GB download would have eaten nearly half of the free disk margin.

UniLLM — studied

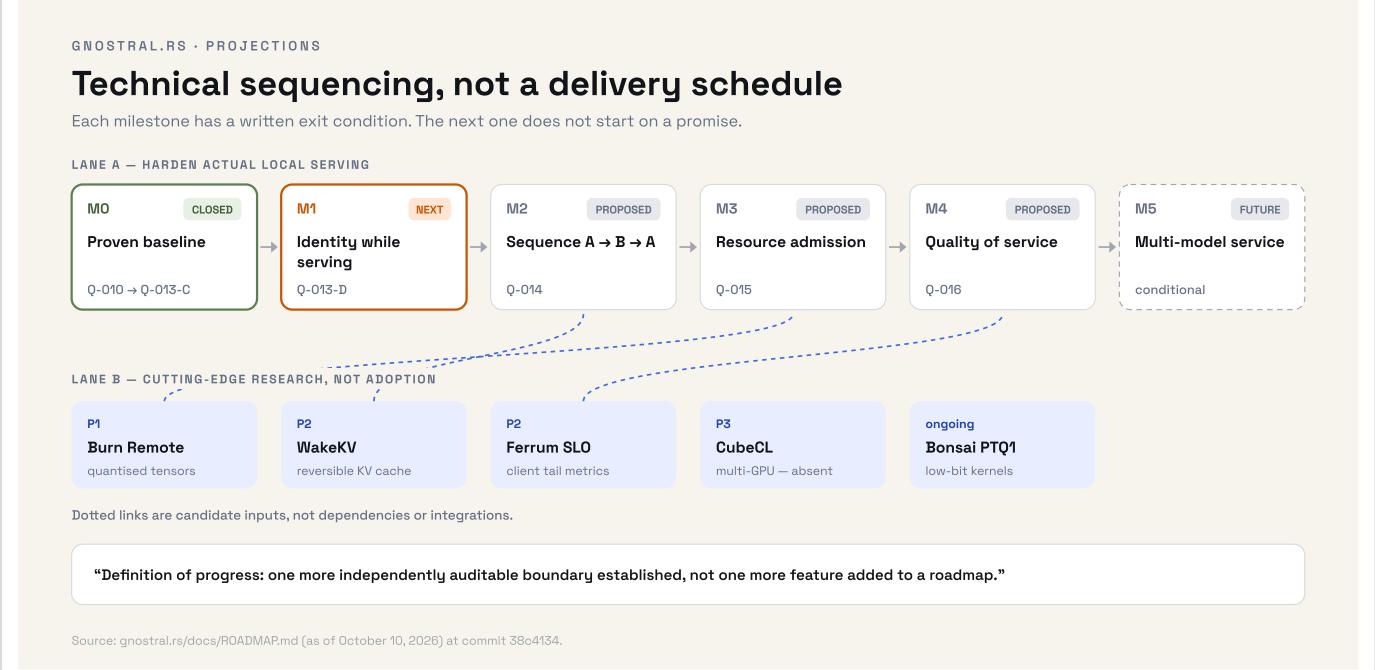
Its registry and lifecycle ideas are adapted; its mocked telemetry is rejected. The rule that came out of it: write "unknown" rather than a plausible value.

A corrected claim

The very first quest, Q-001, measured nothing new: it **withdrew** a figure. A Bonsai benchmark claimed 40 tokens per second; for lack of sufficient evidence, the repository kept 35.2 and never published the 40. So the project starts with a correction — a choice of method.

Projections: the roadmap

The target fits in one sentence: an inspectable, Rust-first, **local** inference control plane for constrained hardware — choosing the right engine for each model, bounding resources, proving readiness, stopping cleanly, accounting for performance. Compute kernels and expert placement stay the business of each engine.



One exit condition per milestone. M1 must prove it is talking to the right process for the whole time it serves; M2, that a model can be loaded, unloaded, another loaded and the first brought back without leaking memory. The multi-model service only comes at the end, and only if everything else holds.

Research, in parallel

A watch on five leads — quantised tensor transport (Burn), client-side latency metrics (Ferrum), a reversible KV cache (WakeKV), several cards (CubeCL), low-bit kernels (Bonsai). Each has its first falsifiable experiment; none is a dependency, and none may silently change the pinned engine.

What will not be promised

No public hosted API, no production-performance claim, no broad portability until the matching gate has passed. A failed run remains first-class evidence, even when a later run succeeds.

What it changes, depending on who you are

Curious about local AI

You understand why a model does not ”fit”, and what each placement choice costs. The dossier’s analogies are enough; the repository provides the details.

Developer

The repository contains a typed engine contract, a bounded process harness, plans tied to a fingerprint and measurement receipts in JSON. Much of it can be tested without a GPU.

Decision-maker or funder

The asset is not a finished product: it is a discipline of evidence, applied to a problem most teams prefer to work around by buying hardware.

Going further

- TECHNICAL — THE REPOSITORY’S WORDS
- CHECKING WITHOUT A GRAPHICS CARD